Week No: 1	Roll Number: 23071A05B0	Date: 2|2|26
Title: Linear Regression without using in-built libraries		

Problem Statement: Implement Linear Regression learning algorithm without using any in-built libraries.

Program:

```
import matplotlib.pyplot as plt
import math
x = [1, 2, 3, 4, 5]
y = [2, 4, 5, 4, 6]

n = len(x)

sum_x = sum(x)
sum_y = sum(y)
sum_xy = sum(x[i] * y[i] for i in range(n))
sum_x2 = sum(x[i] ** 2 for i in range(n))

m = (n * sum_xy - sum_x * sum_y) / (n * sum_x2 - sum_x ** 2)
c = (sum_y - m * sum_x) / n

print("Slope (m):", m)
print("Intercept (c):", c)

def predict(x_val):
    return m * x_val + c

y_pred = [predict(xi) for xi in x]
mse = sum((y[i] - y_pred[i])**2 for i in range(n)) / n
rmse = math.sqrt(mse)

print("Mean Squared Error (MSE):", mse)
```

```
print("Root Mean Squared Error (RMSE):", rmse)


x_test = 6
y_test = predict(x_test)


plt.figure(figsize=(8, 5))


plt.scatter(x, y, label="Actual Data", marker="o")
plt.plot(x, y_pred, label="Regression Line")
plt.scatter(x_test, y_test, label="Prediction Point (x=6)", marker="*")


plt.xlabel("X values")
plt.ylabel("Y values")
plt.title("Linear Regression from Scratch")


plt.legend()
plt.grid(True)
plt.show()
```

Output:

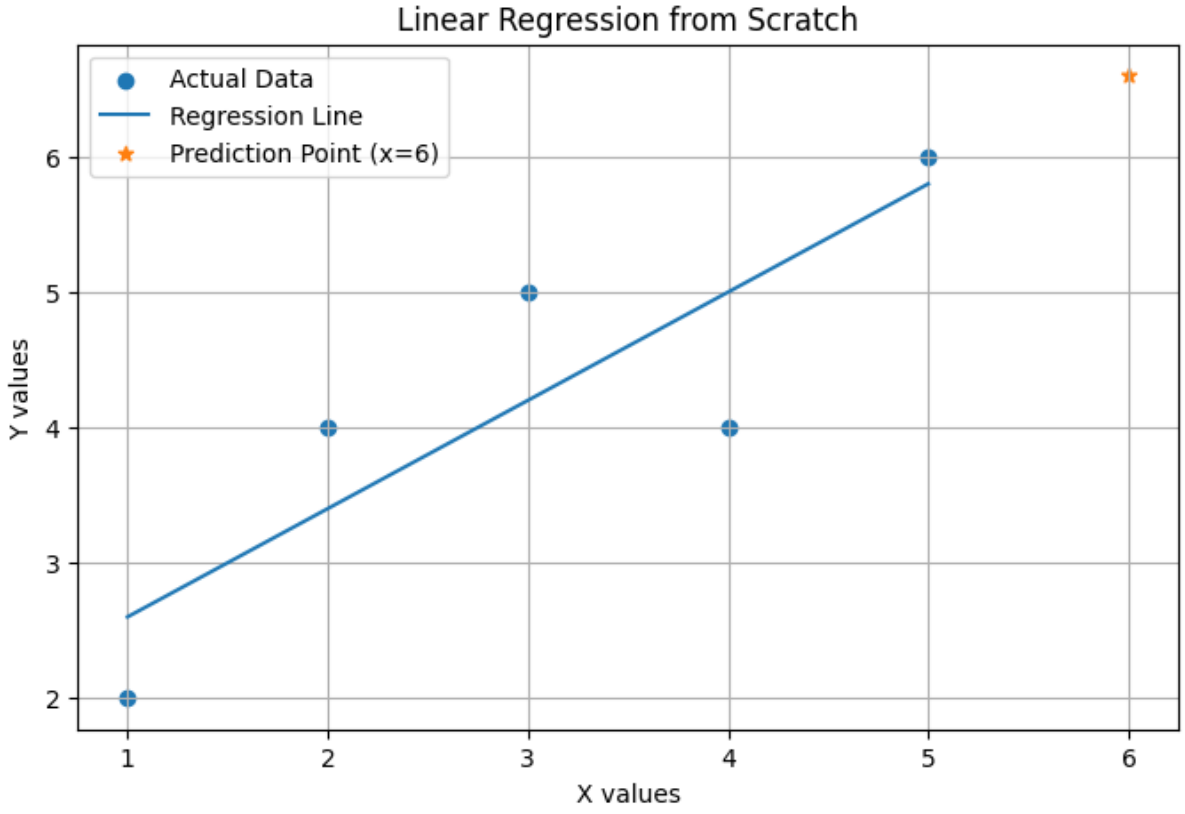
Week No: 2	Roll Number: 23071A05B0	Date: 2|2|26
Title: Linear Regression using in-built libraries		

Slope (m): 0.8

Intercept (c): 1.8

Mean Squared Error (MSE): 0.47999999999999987

Root Mean Squared Error (RMSE): 0.6928203230275508



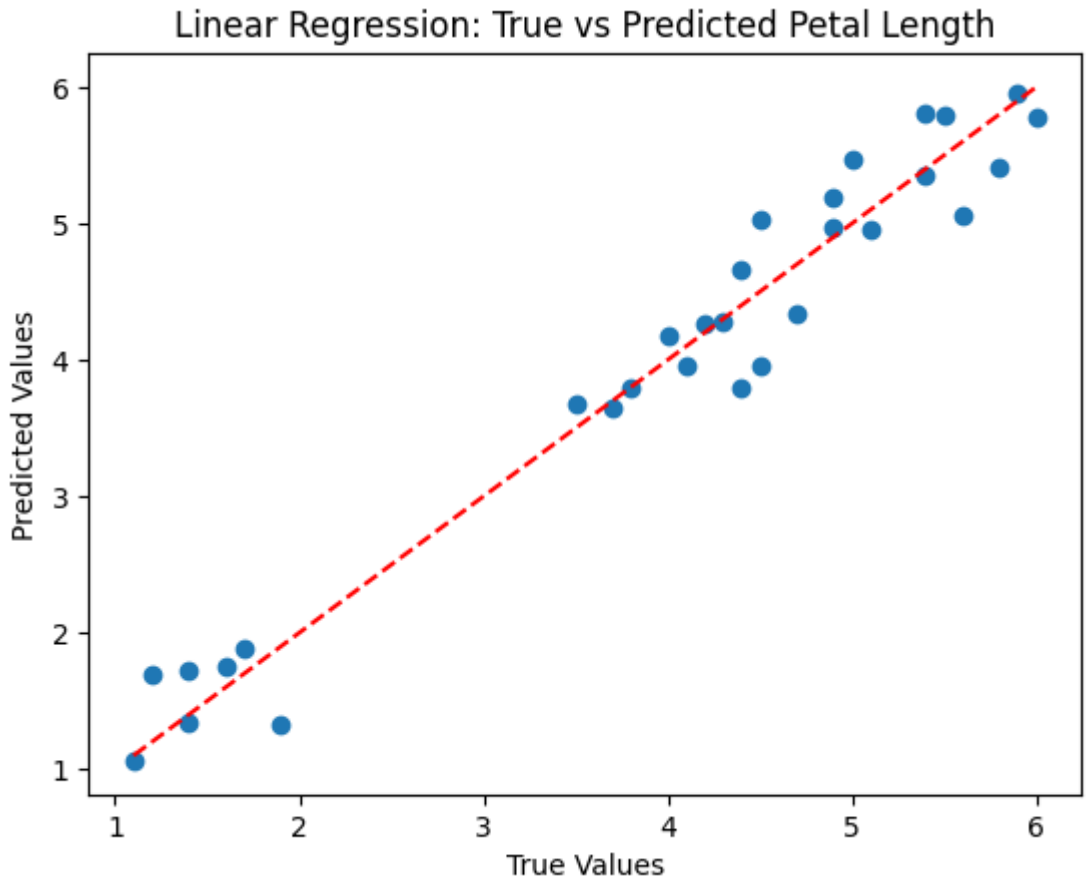
Problem Statement: Implement Linear Regression learning algorithm using in-built libraries

Program:

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split

# Load dataset
data = pd.read_csv("Iris.csv")

# Features and target
X = data[['SepalLengthCm', 'SepalWidthCm', 'PetalWidthCm']]
y = data['PetalLengthCm']

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

# Model
model = LinearRegression()
model.fit(X_train, y_train)

# Prediction
y_pred = model.predict(X_test)

# Plotting true vs predicted values
plt.scatter(y_test, y_pred)
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], color='red', linestyle='--')
plt.xlabel("True Values")
```

```
plt.ylabel("Predicted Values")
plt.title("Linear Regression: True vs Predicted Petal Length")
plt.show()

print("Coefficients:", model.coef_)
print("Intercept:", model.intercept_)
```

Output:



Coefficients: [0.76977217 -0.66842393 1.40631223]

Intercept: -0.3822468155303551

Week No: 3	Roll Number: 23071A05B0	Date: 9|2|26
Title: Implement Naive Bayes		

Problem Statement: Implement Naïve Bayes classifier

Program:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.naive_bayes import GaussianNB
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import accuracy_score

data = pd.read_csv("Iris.csv")

X = data[['SepalLengthCm', 'SepalWidthCm']] # Only first two features for visualization
y = data['Species']

le = LabelEncoder()
y = le.fit_transform(y)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3)

model = GaussianNB()
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

# Print the accuracy
print("Naive Bayes Accuracy:", accuracy_score(y_test, y_pred))
```

Output:

```
Naive Bayes Accuracy: 0.7555555555555555
```

Week No: 4	Roll Number: 23071A05B0	Date: 9|2|26
Title: Decision trees – ID3		

Problem Statement: Implement Decision trees – ID3

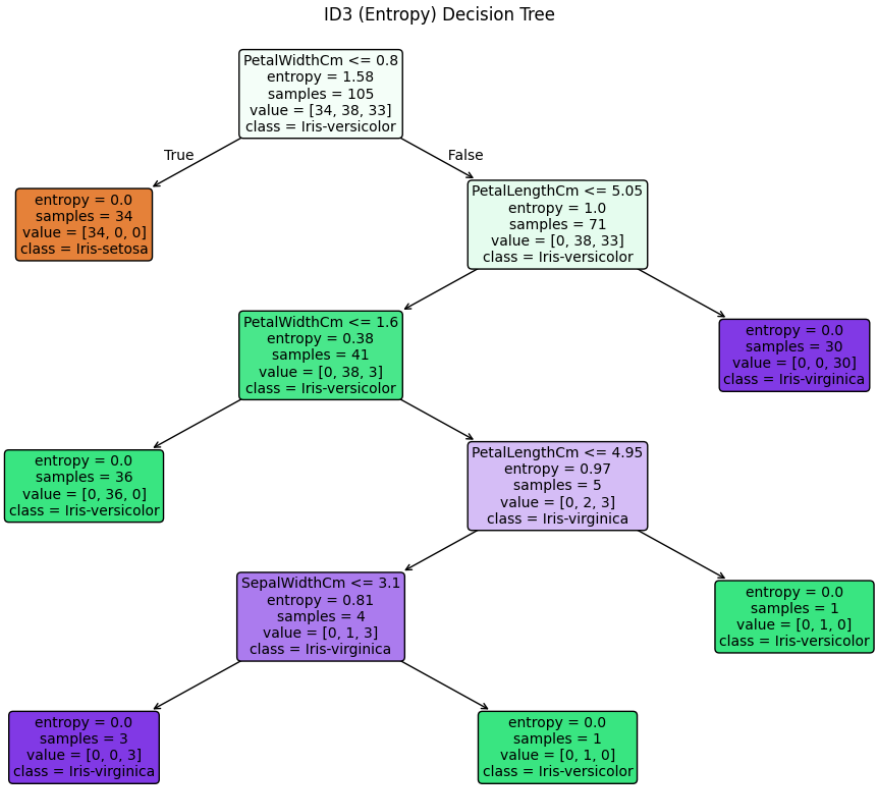
Program:

```
import pandas as pd
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import accuracy_score
import matplotlib.pyplot as plt

data = pd.read_csv("Iris.csv")

X = data[['SepalLengthCm', 'SepalWidthCm', 'PetalLengthCm', 'PetalWidthCm']]
y = data['Species']

le = LabelEncoder()
y = le.fit_transform(y)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3)

model = DecisionTreeClassifier(criterion="entropy")
model.fit(X_train, y_train)
# Prediction on the test set
y_pred = model.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)
print(f"ID3 (Entropy) Accuracy: {accuracy * 100:.2f}%")

plt.figure(figsize=(15,10))
```

```
plot_tree(model, filled=True, feature_names=X.columns, class_names=le.classes_, rounded=True, fontsize=10, precision=2, label='all')

plt.title("ID3 (Entropy) Decision Tree")

plt.show()
```

Output:

ID3 (Entropy) Accuracy: 91.11%



Week No: 5	Roll Number: 23071A05B0	Date: 9|2|26
Title: Decision trees – CART		

Problem Statement: Decision trees – CART

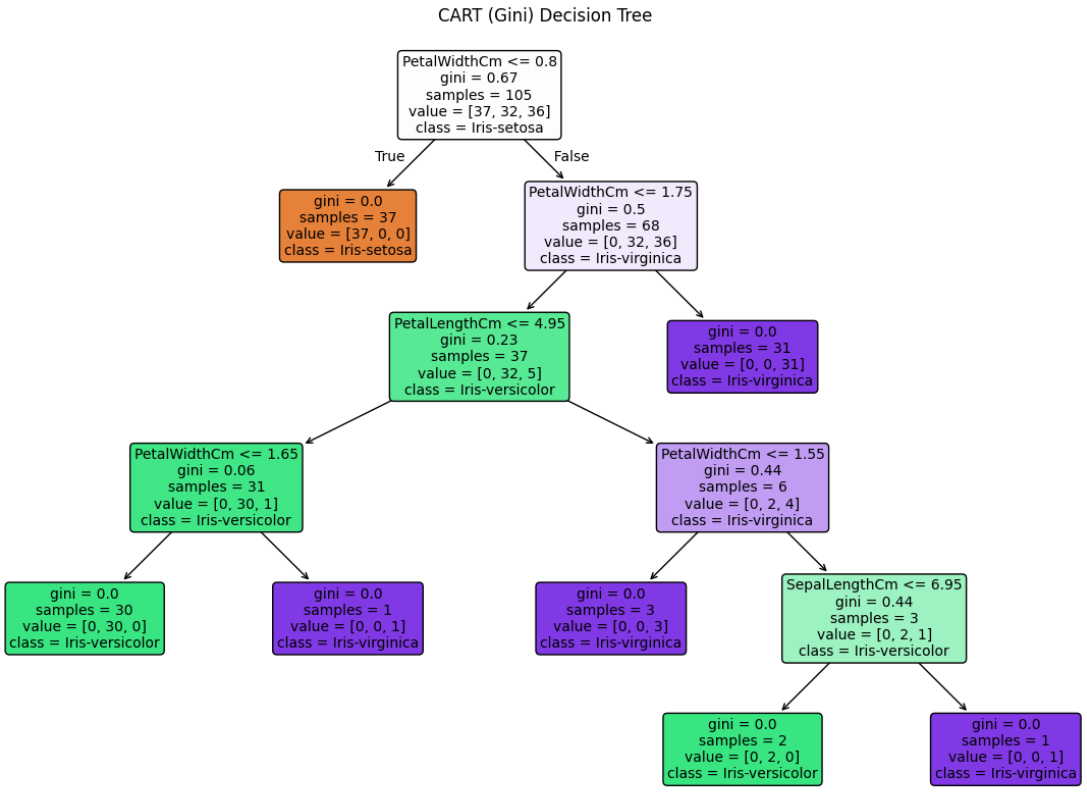
Program:

```
import pandas as pd
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import accuracy_score
import matplotlib.pyplot as plt

data = pd.read_csv("Iris.csv")

X = data[['SepalLengthCm', 'SepalWidthCm', 'PetalLengthCm', 'PetalWidthCm']]
y = data['Species']

le = LabelEncoder()
y = le.fit_transform(y)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3)

model = DecisionTreeClassifier(criterion="gini")
model.fit(X_train, y_train)
# Prediction on the test set
y_pred = model.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)
print(f"CART (Gini) Accuracy: {accuracy * 100:.2f}%")

plt.figure(figsize=(15,10))
```

```
plot_tree(model, filled=True, feature_names=X.columns, class_names=le.classes_, rounded=True, fontsize=10, precision=2, label='all')

plt.title("CART (Gini) Decision Tree")

plt.show()
```

Output:

CART (Gini) Accuracy: 97.78%



Week No: 6	Roll Number: 23071A05B0	Date: 16|2|26
Title: K Nearest Neighbors		

Problem Statement: Implement k-Nearest Neighbor (k-NN)

Program:

```
import pandas as pd
from sklearn.neighbors import KNeighborsClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import cross_val_score

data = pd.read_csv("Iris.csv")

if 'Id' in data.columns:
    data = data.drop(columns=['Id'])

X = data.drop(columns=['Species'])
y = data['Species']

scaler = StandardScaler()
X = scaler.fit_transform(X)
best_k = 1
best_acc = 0

for k in range(1, 11):
    model = KNeighborsClassifier(n_neighbors=k)
    scores = cross_val_score(model, X, y, cv=5)
    acc = scores.mean()
    if acc > best_acc:
        best_acc = acc
        best_k = k
```

```
print("Best K:", best_k)

print("Cross Validation Accuracy:", best_acc)
```

Output:

```
Best K: 6

Cross Validation Accuracy: 0.9666666666666668
```

Week No: 7	Roll Number: 23071A05B0	Date: 30|3|26
Title: Support Vector Machines		

Problem Statement: Implement Support vector machines learning algorithm- Linear and Non - Linear

Program:

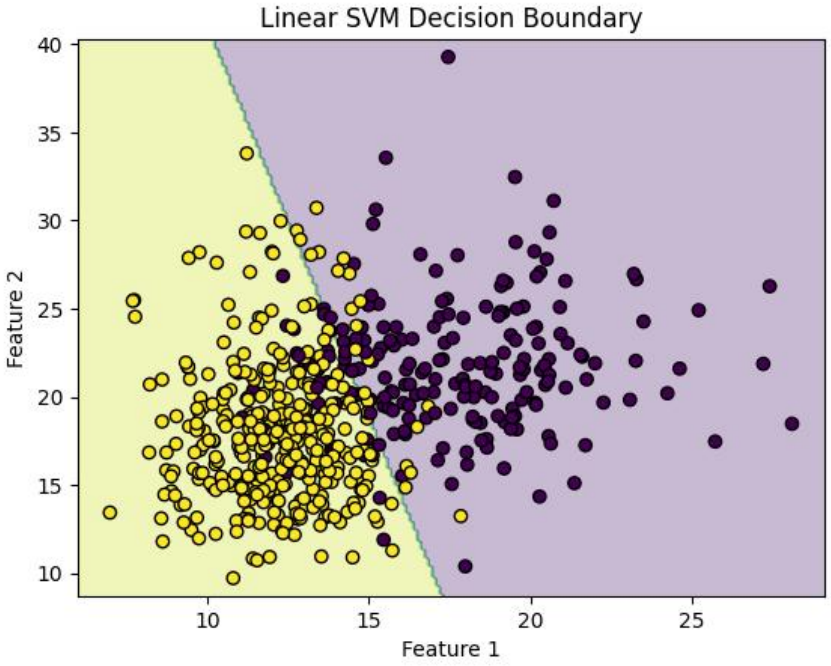
```
import numpy as np
import matplotlib.pyplot as plt
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score
cancer = datasets.load_breast_cancer()

X = cancer.data[:, :2]
y = cancer.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

linear_svm = SVC(kernel='linear')
linear_svm.fit(X_train, y_train)


y_pred_linear = linear_svm.predict(X_test)
linear_accuracy = accuracy_score(y_test, y_pred_linear)
print("Linear SVM Accuracy:", linear_accuracy)


rbf_svm = SVC(kernel='rbf')
```

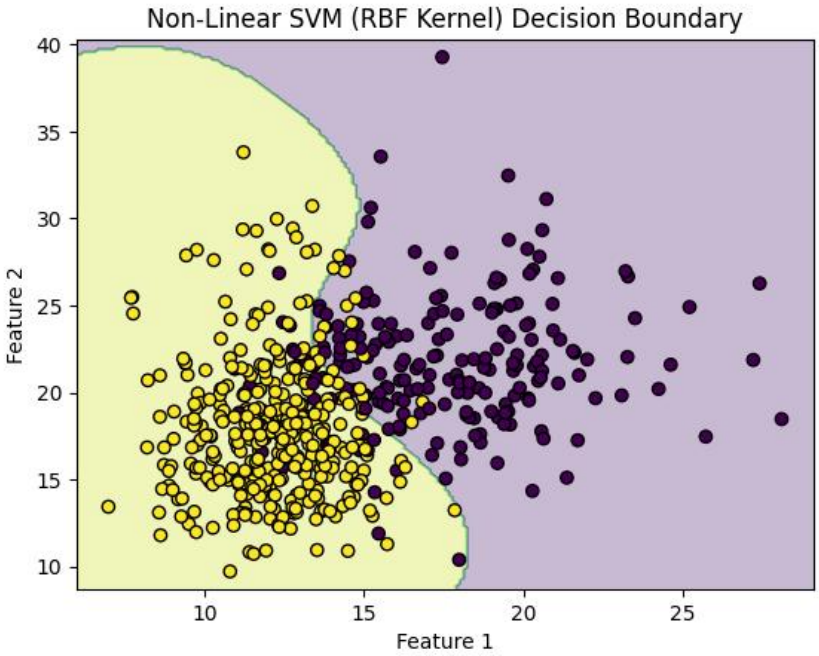
```
rbf_svm.fit(X_train, y_train)

y_pred_rbf = rbf_svm.predict(X_test)

rbf_accuracy = accuracy_score(y_test, y_pred_rbf)
print("Non-Linear (RBF) SVM Accuracy:", rbf_accuracy)

def plot_decision_boundary(model, X, y, title):

    x_min, x_max = X[:, 0].min() - 1, X[:, 0].max() + 1
    y_min, y_max = X[:, 1].min() - 1, X[:, 1].max() + 1

    xx, yy = np.meshgrid(
        np.linspace(x_min, x_max, 200),
        np.linspace(y_min, y_max, 200)
    )

    Z = model.predict(np.c_[xx.ravel(), yy.ravel()])
    Z = Z.reshape(xx.shape)

    plt.contourf(xx, yy, Z, alpha=0.3)
    plt.scatter(X[:, 0], X[:, 1], c=y, edgecolors='k')
    plt.title(title)
    plt.xlabel("Feature 1")
    plt.ylabel("Feature 2")
    plt.show()

plot_decision_boundary(linear_svm, X, y, "Linear SVM Decision Boundary")
plot_decision_boundary(rbf_svm, X, y, "Non-Linear SVM (RBF Kernel) Decision Boundary")
```

Output:

Linear SVM Accuracy: 0.9035087719298246

Non-Linear (RBF) SVM Accuracy: 0.9210526315789473





Week No: 8	Roll Number: 23071A05B0	Date: 6|4|26
Title: K-means algorithm		

Problem Statement: Implement unsupervised k-means algorithm

Program:

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans

df = pd.read_csv("iris.csv")
X = df.select_dtypes(include=['float64', 'int64'])
kmeans = KMeans(n_clusters=3, random_state=42)
kmeans.fit(X)

df['Cluster'] = kmeans.labels_
centroids = kmeans.cluster_centers_

print("Centroids:\n", centroids)

x_axis = X.iloc[:, 0]
y_axis = X.iloc[:, 1]

plt.figure()
plt.scatter(x_axis, y_axis, c=df['Cluster'])
plt.scatter(centroids[:, 0], centroids[:, 1], marker='X', s=200)
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.title("K-Means Clustering on Iris Dataset")

plt.show()
```

Output:

Centroids:

[[74.5 5.922 2.78 4.206 1.304]
[125. 6.57058824 2.97058824 5.52352941 2.01176471]
[25. 5.00612245 3.42040816 1.46530612 0.24489796]]

Week No: 9	Roll Number: 23071A05B0	Date: 6|4|26
Title: EM Algorithm Vs. K-means Algorithm		

Problem Statement: Apply EM algorithm to cluster a set of data stored in a .CSV file. Use the same dataset for clustering using k-Means algorithm. Compare the results of these two algorithms and comment on the quality of clustering.

Program:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.cluster import KMeans
from sklearn.mixture import GaussianMixture
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import silhouette_score, adjusted_rand_score

df = pd.read_csv("iris.csv")

X = df.select_dtypes(include=[np.number])

true_labels = None
if 'species' in df.columns:
    true_labels = df['species'].astype('category').cat.codes

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

kmeans = KMeans(n_clusters=3, random_state=42)
kmeans_labels = kmeans.fit_predict(X_scaled)

gmm = GaussianMixture(n_components=3, random_state=42)
gmm_labels = gmm.fit_predict(X_scaled)

kmeans_silhouette = silhouette_score(X_scaled, kmeans_labels)
gmm_silhouette = silhouette_score(X_scaled, gmm_labels)

print("----- Clustering Evaluation -----")
print(f"K-Means Silhouette Score: {kmeans_silhouette:.4f}")
print(f"EM (GMM) Silhouette Score: {gmm_silhouette:.4f}")
```

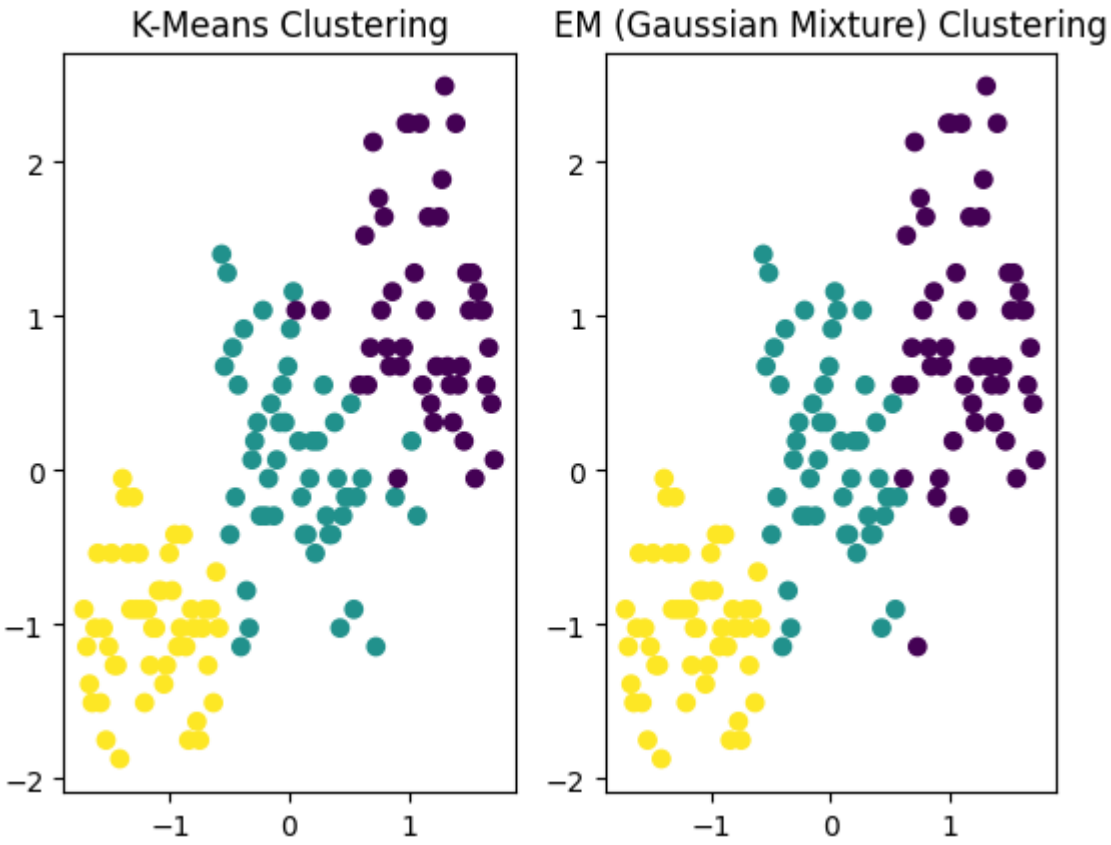
```
if true_labels is not None:
    kmeans_ari = adjusted_rand_score(true_labels, kmeans_labels)
    gmm_ari = adjusted_rand_score(true_labels, gmm_labels)

    print(f"K-Means ARI: {kmeans_ari:.4f}")
    print(f"EM (GMM) ARI: {gmm_ari:.4f}")

plt.figure()

# Plot K-Means
plt.subplot(1, 2, 1)
plt.scatter(X_scaled[:, 0], X_scaled[:, 1], c=kmeans_labels)
plt.title("K-Means Clustering")

# Plot EM (GMM)
plt.subplot(1, 2, 2)
plt.scatter(X_scaled[:, 0], X_scaled[:, 1], c=gmm_labels)
plt.title("EM (Gaussian Mixture) Clustering")

plt.show()

print("\n----- Conclusion -----")

if gmm_silhouette > kmeans_silhouette:
    print("EM (GMM) produced better-defined clusters than K-Means.")
else:
    print("K-Means performed as well as or better than EM.")
```

Output:

----- Clustering Evaluation -----

K-Means Silhouette Score: 0.4529

EM (GMM) Silhouette Score: 0.4425



----- Conclusion -----

K-Means performed as well as or better than EM.

Week No: 10	Roll Number: 23071A05B0	Date: 6|4|26
Title: Single Layer Perceptron		

Problem Statement: Implement a single layer perceptron

Program:

```
import pandas as pd
from sklearn.linear_model import Perceptron
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

df = pd.read_csv("iris.csv")
df = df[df['Species'] != 'setosa']

X = df.drop(columns=['Species'])
y = df['Species']
y = y.astype('category').cat.codes

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

model = Perceptron(max_iter=1000, eta0=0.1, random_state=42)
model.fit(X_train, y_train)

y_pred = model.predict(X_test)
```

```
accuracy = accuracy_score(y_test, y_pred)


print("Accuracy:", round(accuracy, 4))
print("Weights:", model.coef_)
print("Bias:", model.intercept_)
```

Output:

```
Accuracy: 0.9667
Weights: [[-0.08506113 -0.08421045  0.21539413 -0.16471323 -0.13561319]
 [-0.46127523  0.03992124 -0.45609567  0.583879   -0.18026224]
 [ 0.38223476  0.00446955  0.05173958  0.22993909  0.23057574]]
Bias: [-0.1 -0.3 -0.4]
```

Week No: 11	Roll Number: 23071A05B0	Date: 6|4|26
Title: Random Forrest Algorithm		

Problem Statement: Implement Random Forest model learning algorithm

Program:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

df = pd.read_csv("iris.csv")
print("Columns in Dataset:")
print(df.columns)

if "Id" in df.columns:
    df = df.drop("Id", axis=1)
if "Unnamed: 0" in df.columns:
    df = df.drop("Unnamed: 0", axis=1)

print("\nUpdated Columns:")
print(df.columns)

X = df.iloc[:, :-1]
y = df.iloc[:, -1]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

```
print("\nAccuracy:", accuracy_score(y_test, y_pred))
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))
print("\nClassification Report:")
print(classification_report(y_test, y_pred))

sample = pd.DataFrame(
    [[5.1, 3.5, 1.4, 0.2]],
    columns=X.columns
)

prediction = model.predict(sample)
print("\nPrediction for sample is:", prediction[0])
```

Output:

```
Columns in Dataset:
Index(['Id', 'SepalLengthCm', 'SepalWidthCm', 'PetalLengthCm', 'PetalWidthCm',
       'Species'],
      dtype='object')

Updated Columns:
Index(['SepalLengthCm', 'SepalWidthCm', 'PetalLengthCm', 'PetalWidthCm',
       'Species'],
      dtype='object')

Accuracy: 1.0

Confusion Matrix:
[[19  0  0]
```

[0 13 0]

[0 0 13]]

Classification Report:

	precision	recall	f1-score	support
Iris-setosa	1.00	1.00	1.00	19
Iris-versicolor	1.00	1.00	1.00	13
Iris-virginica	1.00	1.00	1.00	13
accuracy			1.00	45
macro avg	1.00	1.00	1.00	45
weighted avg	1.00	1.00	1.00	45

Prediction for sample is: Iris-setosa

Week No: 12	Roll Number: 23071A05B0	Date: 13|4|26
Title: Bagging Algorithm		

Problem Statement: Implement Bagging algorithm

Program:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import BaggingClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

df = pd.read_csv("iris.csv")
if "Id" in df.columns:
    df = df.drop("Id", axis=1)
if "Unnamed: 0" in df.columns:
    df = df.drop("Unnamed: 0", axis=1)

print("Dataset Preview:")
print(df.head())

X = df.iloc[:, :-1]
y = df.iloc[:, -1]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
base_model = DecisionTreeClassifier()

model = BaggingClassifier(
    estimator=base_model,
    n_estimators=10,
```

```
    random_state=42
)

model.fit(X_train, y_train)
y_pred = model.predict(X_test)
print("\nAccuracy:", accuracy_score(y_test, y_pred))
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))
print("\nClassification Report:")
print(classification_report(y_test, y_pred))

sample = pd.DataFrame(
    [[5.1, 3.5, 1.4, 0.2]],
    columns=X.columns
)

prediction = model.predict(sample)
print("\nPrediction for sample is:", prediction[0])
```

Output:

Dataset Preview:

	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species
0	5.1	3.5	1.4	0.2	Iris-setosa
1	4.9	3.0	1.4	0.2	Iris-setosa
2	4.7	3.2	1.3	0.2	Iris-setosa
3	4.6	3.1	1.5	0.2	Iris-setosa
4	5.0	3.6	1.4	0.2	Iris-setosa

Accuracy: 1.0

Confusion Matrix:

[[19 0 0]

[0 13 0]

[0 0 13]]

Classification Report:

	precision	recall	f1-score	support
Iris-setosa	1.00	1.00	1.00	19
Iris-versicolor	1.00	1.00	1.00	13
Iris-virginica	1.00	1.00	1.00	13
accuracy			1.00	45
macro avg	1.00	1.00	1.00	45
weighted avg	1.00	1.00	1.00	45

Prediction for sample is: Iris-setosa

Week No: 13	Roll Number: 23071A05B0	Date: 20|4|26
Title: Boosting Algorithms		

Problem Statement: Implement Adaboost, XGboost algorithms.

Program:

Adaboost

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import AdaBoostClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

df = pd.read_csv("iris.csv")

if "Id" in df.columns:
    df = df.drop("Id", axis=1)
if "Unnamed: 0" in df.columns:
    df = df.drop("Unnamed: 0", axis=1)

print("Dataset Preview:")
print(df.head())

X = df.iloc[:, :-1]
y = df.iloc[:, -1]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
base_model = DecisionTreeClassifier(max_depth=1)
model = AdaBoostClassifier(
    estimator=base_model,
```

```
    n_estimators=50,
    learning_rate=1.0,
    random_state=42
)

model.fit(X_train, y_train)
y_pred = model.predict(X_test)
print("\nAccuracy:", accuracy_score(y_test, y_pred))

print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))

print("\nClassification Report:")
print(classification_report(y_test, y_pred))

sample = pd.DataFrame(
    [[5.1, 3.5, 1.4, 0.2]],
    columns=X.columns
)

prediction = model.predict(sample)
print("\nPrediction for sample is:", prediction[0])
```

XGboost

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
from xgboost import XGBClassifier
```

```
df = pd.read_csv("iris.csv")

if "Id" in df.columns:
    df = df.drop("Id", axis=1)
if "Unnamed: 0" in df.columns:
    df = df.drop("Unnamed: 0", axis=1)

print("Dataset Preview:")
print(df.head())

X = df.iloc[:, :-1]
y = df.iloc[:, -1]

le = LabelEncoder()
y = le.fit_transform(y)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

model = XGBClassifier(
    n_estimators=100,
    learning_rate=0.1,
    max_depth=3,
    objective="multi:softmax",
    num_class=3,
    random_state=42
)

model.fit(X_train, y_train)
```

```
y_pred = model.predict(X_test)

print("\nAccuracy:", accuracy_score(y_test, y_pred))

print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))

print("\nClassification Report:")
print(classification_report(y_test, y_pred))

sample = pd.DataFrame(
    [[5.1, 3.5, 1.4, 0.2]],
    columns=X.columns
)

prediction = model.predict(sample)
print("\nPrediction for sample is:", le.inverse_transform(prediction)[0])
```

Output:

Adaboost

Dataset Preview:

	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species
0	5.1	3.5	1.4	0.2	Iris-setosa
1	4.9	3.0	1.4	0.2	Iris-setosa
2	4.7	3.2	1.3	0.2	Iris-setosa
3	4.6	3.1	1.5	0.2	Iris-setosa
4	5.0	3.6	1.4	0.2	Iris-setosa

Accuracy: 1.0

Confusion Matrix:

[[19 0 0]
[0 13 0]
[0 0 13]]

Classification Report:

	precision	recall	f1-score	support
Iris-setosa	1.00	1.00	1.00	19
Iris-versicolor	1.00	1.00	1.00	13
Iris-virginica	1.00	1.00	1.00	13
accuracy			1.00	45
macro avg	1.00	1.00	1.00	45
weighted avg	1.00	1.00	1.00	45

Prediction for sample is: Iris-setosa

XGboost

Dataset Preview:

	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species
0	5.1	3.5	1.4	0.2	Iris-setosa
1	4.9	3.0	1.4	0.2	Iris-setosa
2	4.7	3.2	1.3	0.2	Iris-setosa
3	4.6	3.1	1.5	0.2	Iris-setosa
4	5.0	3.6	1.4	0.2	Iris-setosa

Accuracy: 1.0

Confusion Matrix:

[[19 0 0]
[0 13 0]
[0 0 13]]

Classification Report:

	precision	recall	f1-score	support
0	1.00	1.00	1.00	19
1	1.00	1.00	1.00	13
2	1.00	1.00	1.00	13
accuracy			1.00	45
macro avg	1.00	1.00	1.00	45
weighted avg	1.00	1.00	1.00	45

Prediction for sample is: Iris-setosa